Organizzazione del corso, Git e IA

4TI · lezione del 29/09/2026

Versione 1.0 · 29/10/2026

Corso a cura del prof. Nicola Regge

Che cosa abbiamo fatto

1. **Quando:** 29/09/2026, 3 ore, Laboratorio.
2. **Argomenti:** Organizzazione del corso; Git, GitHub e l'intelligenza artificiale (NotebookLM) per studiare.

La teoria e l'esercitazione

Documento di teoria per l'allievo, in versione testo (.md). E la stessa dispensa in formato testo puro: puoi darla a Gemini per farti spiegare o tradurre le parti difficili, o caricarla come fonte. I disegni (i grafi dei rami) qui sono descritti a parole. Regola d'oro del corso: l'IA serve a CAPIRE meglio, non a copiare; la prova del nove e saperlo spiegare a parole tue.

---

1. I tre strumenti e come si legano

1. Usiamo tre ambienti diversi, ognuno con un compito preciso:
2. **Git e GitHub**: il magazzino ordinato del lavoro, dove ogni versione resta salvata e condivisibile.
3. **Gemini**: l'IA (Intelligenza Artificiale) generativa a cui si parla; le si da un testo (anche un file .md) e ti spiega, traduce, riassume.
4. **NotebookLM**: un tutor che risponde SOLO sui documenti che carichi (per esempio un PDF), quindi resta fedele alla fonte.
5. Idea da tenere a mente: **Git conserva**, **Gemini spiega**, **NotebookLM ripassa sulla fonte**. Non sono concorrenti: si usano insieme.

2. Git e GitHub

1. **Git** e un sistema di **versionamento**: tiene la storia del lavoro, cosi ogni salvataggio importante resta e si puo tornare indietro.
2. **GitHub** e il sito in rete dove il lavoro versionato vive e si condivide: e "Git su internet". Attenzione a non confonderli: **Git** e lo strumento (il motore), **GitHub** e il servizio online (il garage online).

2.1 GitHub: un'azienda e un servizio

1. E un'**azienda** (GitHub, Inc., dal 2018 di Microsoft): come ogni azienda deve mantenersi.

2. E un **servizio** online: un sito dove metti i progetti in rete, li tieni al sicuro e ci lavori in gruppo, tutto da browser.

2.2 Il repository, il commit, il branch

1. Un **repository** (o “repo”) e la cartella-progetto con dentro i file e tutta la loro storia.
2. Un **commit** e un salvataggio con etichetta: fotografa il progetto in quel momento e gli mette un messaggio (“cosa ho cambiato”). Tanti commit in fila fanno la **storia**.
3. Un **branch** (ramo) e una strada di lavoro parallela, per provare senza rovinare la versione principale (il **main**); riunire un ramo al main si chiama **merge**.
4. La **Pull Request** (PR) e la proposta di unire il proprio ramo nella versione principale, dopo che qualcuno l’ha controllata.

2.3 Che tipo di servizio: PaaS e Freemium

1. **PaaS** (Platform as a Service: piattaforma come servizio): qualcuno ti mette online una piattaforma gia pronta e tu la usi senza costruirti server e computer. Fa parte della famiglia “…aaS” (IaaS = server nudi, PaaS = piattaforma pronta, SaaS = software pronto nel browser).
2. **Freemium** (free + premium): le funzioni di base sono gratis; per le funzioni avanzate si paga. A te studente basta la parte gratis.

3. Il versioning: le versioni 1.2.3

1. Quando un progetto raggiunge un punto “buono”, gli si da un numero di versione di tre cifre `1.2.3`:
2. **major** (la prima cifra): aumenta quando cambi tutto in modo grosso, magari incompatibile col vecchio.
3. **minor** (la seconda): aumenta quando aggiungi una funzione nuova, ma il resto funziona ancora.
4. **patch** (la terza): aumenta quando correggi solo un bug, niente di nuovo.
5. Esempio: `v1.0.0` prima versione giocabile, aggiungi i suoni `v1.1.0`, correggi un bug `v1.1.1`, rifai tutto da capo `v2.0.0`.
6. Una versione pubblicata (**release**) e “congelata”.

> **NOTA**
>
> Regola d’oro del versioning: una versione gia pubblicata non si riusa mai. Se cambi anche una sola cosa, prima aumenti il numero e poi pubblichi.

4. Casi d’uso di Git (i grafi, descritti a parole)

1. Sviluppo lineare: un solo autore, nessun ramo. I commit vanno in fila, la storia e una linea dritta. Se qualcosa si rompe, torni indietro di un commit.

2. Una funzione su un branch, poi merge: provi una funzione su un ramo separato mentre il main resta sano; quando e pronta la unisci (merge).

3. Due funzioni in parallelo: due rami partono insieme e si fondono in tempi diversi; il main raccoglie i pezzi man mano.

4. Hotfix urgente: durante una funzione lunga esce un bug grave; apri un ramo hotfix corto, lo correggi e lo unisci subito, poi torni alla tua funzione.

5. Le release: ogni tanto pubblichi una versione (bandierina sul main): v1.0, v1.1, v1.2… fino a un cambiamento grosso v2.0. Ogni release e congelata.

6. Ramo abbandonato: provi un’idea su un ramo e non la unisci mai; il main non ne risente. Nessun danno.

7. Ramo da un altro ramo: una funzione grande stacca un sotto-ramo; prima rientra il sotto-ramo nel ramo, poi il ramo nel main (scatole dentro scatole).

8. Grande integrazione (gruppo): ognuno ha il suo ramo (grafica, movimenti, audio, menu) e alla fine confluiscono tutti nel main con le Pull Request: nasce la v1.0.

9. Ramo di manutenzione: il main va verso la v2.0, ma per chi usa ancora la v1 tieni un ramo parallelo con solo i fix (v1.1, v1.2).

10. Revert: un commit ha introdotto un problema; non cancelli la storia, aggiungi un nuovo commit che annulla quello sbagliato (revert), lasciando traccia onesta.

11. Conflitto di merge: due rami cambiano la stessa riga in modo diverso; Git si ferma e una persona sceglie cosa tenere, chiudendo con un commit di risoluzione.

12. Squadra a 5 rami con due release: sul main si pubblicano v1.0 e v2.0 mentre quattro rami lavorano in parallelo e rientrano in momenti diversi.

13. Rami annidati a 5 livelli: un lavoro grande si divide in rami dentro rami, fino a cinque livelli; ogni livello rientra nel precedente, come le matriosche.

14. **Due major insieme**: dopo la v1.0 il main punta alla v2.0, ma restano vivi due rami di manutenzione (v1.x e poi v2.x).

15. **Storia realistica**: tutto insieme — release, feature con sotto-ramo, hotfix, revert e un merge con conflitto: la fotografia di un progetto vero.

> **NOTA**
>
> Come si legge un grafo: ogni pallino e un commit, la linea orizzontale in alto e il main, le linee colorate che si staccano sono branch, quando due linee si riuniscono e un merge, le bandierine gialle sono le release.

5. I file Markdown (.md)

1. Un file **Markdown** (estensione **.md**) e **testo puro** con pochi simboli:
2. `#` a inizio riga fa un titolo;
3. `**parola**` mette in grassetto;
4. una riga con `1.` fa un elenco numerato.
5. Si legge ovunque e non contiene immagini pesanti: solo contenuto. Per questo l'IA lo legge molto bene: nel corso diamo le dispense anche in .md proprio per darle a Gemini.

6. Gemini: l'IA generativa, e come usarla con un file .md

1. **Gemini** e un'IA **generativa**: le scrivi una richiesta (un "prompt") e genera una risposta: spiega, traduce, riassume, fa domande di ripasso.
2. L'IA puo **sbagliare o inventare** (allucinazioni): quello che dice va controllato, soprattutto date, numeri e fonti.

6.1 Passi per usare Gemini con un file .md

1. Nel browser vai su questo indirizzo, con l'account della scuola:

```
gemini.google.com
```

1. Apri il file .md, **seleziona tutto** e **copialo**; torna su Gemini e **incolla** (oppure allega il file).
2. Scrivi la richiesta, per esempio:

```
Spiegami con parole semplici la parte che non capisco: [incolla la frase].
```

6.2 Quale versione di Gemini scegliere

1. In alto Gemini ha un menu per la **versione del modello**. I nomi/numeri cambiano; conta a cosa serve ciascuna:
2. **Flash** (veloce): risposte rapide per cose semplici (spiegazioni, traduzioni, riassunti brevi). La scelta di tutti i giorni.
3. **Pro** (per esempio “3.1 Pro”, piu potente): per compiti complessi e testi lunghi. Piu lenta ma piu capace.
4. **Ragionamento** (per esempio “3.6 ragionamento”, che “pensa” prima di rispondere): per problemi difficili, passo per passo. La piu lenta ma la piu accurata.
5. Regola pratica: parti da **Flash**; se la risposta e complessa o ti sembra sbagliata, passa a **Pro** o **Ragionamento** e richiedi.
6. Guarda a **cosa serve** la versione (veloce / potente / ragiona), non solo al numero.

> **NOTA**
> Le versioni piu potenti o di “ragionamento”, con l’account gratuito, possono avere un limite giornaliero: usale quando servono davvero, per il resto va bene Flash.

7. NotebookLM: il tutor sulle fonti, e come usarlo con un PDF

1. **NotebookLM** risponde **solo** sui documenti che carichi: se carichi un PDF, ti risponde usando quel PDF e non inventa fuori da li.
2. E ottimo per **studiare e ripassare** su una fonte precisa (una dispensa, un capitolo): resta fedele al testo.

7.1 Passi per usare NotebookLM con un PDF

1. Nel browser vai su questo indirizzo, con l’account della scuola:

```
notebooklm.google.com
```

1. Clicca `Crea nuovo`, poi `Carica`, e scegli un file **PDF**; aspetta che la fonte compaia a sinistra.
2. Nel riquadro della chat scrivi, per esempio:

```
Fammi un riassunto semplice di questo documento.
```

> **NOTA**
>
> Se non hai un PDF sul computer: su github.com entra nella cartella del documento, clicca sul PDF e poi sull'icona di download (freccia in giu) in alto a destra.

8. Il flusso completo: i tre ambienti insieme

1. Il materiale del corso sta su **Git/GitHub** (versionato).
2. Da li si **scarica** il PDF (per leggere/stampare) o il file **.md** (per l'IA).
3. Il **.md** si da a **Gemini** per farsi spiegare o tradurre.
4. Il **PDF** si carica su **NotebookLM** per interrogarlo e ripassare restando sulla fonte.
5. Il lavoro finito si **consegna su Classroom**.

Strumento	A cosa serve	Cosa gli dai
Git / GitHub	Conservare e condividere il lavoro versionato	I file del progetto
Gemini	Spiegare, tradurre, riassumere	Un testo o un file .md
NotebookLM	Ripassare restando fedele alla fonte	Un documento (per esempio un PDF)

9. Uso responsabile dell'IA

1. **Capire, non copiare**: l'IA e un aiuto per imparare, non un modo per saltare il lavoro. La prova del nove: saperlo spiegare a parole proprie.
2. **Controllare sempre**: l'IA puo sbagliare o inventare; verifica quello che dice.
3. **Privacy e dati**: non inserire dati personali, di compagni o di famiglia negli assistenti.

10. Prendi appunti a mano

1. Prima di usare gli strumenti, disegna sul quaderno tre riquadri (Git, Gemini, NotebookLM) e, per ognuno, a cosa serve e cosa gli dai.
2. Gli appunti a mano confluiscono nel tuo quaderno personale: aiutano a fissare i concetti e servono per la prova del nove.

Compito unico in tre parti. Provi le tre cose viste in classe: capire e usare Git/GitHub, usare Gemini con un file .md, usare NotebookLM con un PDF. Puoi tenere aperta la dispensa di teoria. Si lavora tutto da browser, senza installare nulla. Alla fine consegni su Classroom.

---

1. Come si consegna

1. Scrivi le risposte in un documento di **Google Documenti** (uno solo per tutto il compito).
2. Intitola il documento con **Cognome e Nome** e **“Compito Git e AI”**.
3. Dove chiesto, incolla uno **screenshot** (cattura schermo) come prova.
4. Alla fine **consegni** il documento nel compito su **Classroom**.

> **NOTA**
>
> Come si fa uno screenshot (Windows 10 e 11): premi insieme i tasti `Windows` + `Maiusc` + `S`, seleziona la parte di schermo, poi incolla nel documento con `Ctrl` + `V`.

2. Parte 1 — Git e GitHub

2.1 Domande (rispondi a parole tue)

1. Con parole tue, che differenza c’e tra **Git** e **GitHub**?
2. Che cos’e un **repository**?
3. Che cos’e un **commit** e a cosa serve avere la storia dei commit?
4. A cosa serve un **branch** e cosa propone una **Pull Request**?

2.2 Azione pratica

1. Nel browser entra su questo indirizzo con il tuo account:

```
github.com
```

1. Apri un **repository** del corso (quello indicato dal docente) e guarda l’elenco dei file.
2. Apri la sezione dei **commit** (la storia) e osserva gli ultimi salvataggi.
3. Fai uno **screenshot** della storia dei commit e incollalo nel documento.

4. Scrivi due righe: **cosa** mostra quella schermata (che cosa e cambiato nell'ultimo commit).

3. Parte 2 — Gemini con un file .md

3.1 Esercitazione

1. Nel browser entra su questo indirizzo con l'account della scuola:

```
gemini.google.com
```

1. In alto scegli la versione **Flash** (quella veloce): per questo compito basta.
2. Apri il file **.md** indicato dal docente (per esempio la dispensa di teoria in formato .md), **seleziona tutto** e **copialo**.
3. **Incolla** il testo in Gemini e chiedi, con parole tue, di spiegarti una parte. Puoi partire da qui:

```
Spiegami con parole semplici cosa sono i file Markdown e perche sono comodi per l'IA.
```

1. Poi chiedi a Gemini di verificarti:

```
Fammi 3 domande per controllare se ho capito, poi correggimi le risposte.
```

3.2 Cosa consegni della Parte 2

1. Uno **screenshot** della risposta di Gemini.
2. Con parole tue: **cosa** ti ha spiegato e se hai dovuto **correggere** qualcosa (ricorda: l'IA puo sbagliare).
3. Scrivi quale **versione** hai usato (Flash / Pro / ragionamento) e perche.

4. Parte 3 — NotebookLM con un PDF

4.1 Esercitazione

1. Nel browser entra su questo indirizzo con l'account della scuola:

```
notebooklm.google.com
```

1. Clicca `Crea nuovo`, poi `Carica`, e scegli il **PDF** indicato dal docente (per esempio la dispensa di teoria in PDF).
2. Aspetta che la fonte compaia a sinistra: e pronta.
3. Nel riquadro della chat scrivi:

```
Fammi un riassunto semplice di questo documento.
```

1. Poi chiedi:

```
Fammi 3 domande a risposta multipla su questo documento.
```

4.2 Cosa consegni della Parte 3

1. Uno **screenshot** del riassunto e delle domande generate.
2. Con parole tue: qual e la **differenza** che hai notato tra chiedere a **Gemini** e chiedere a **NotebookLM**?

5. Domanda finale (fa la differenza)

1. In tre righe, spiega il **flusso completo**: dove sta il materiale, cosa dai a Gemini, cosa carichi su NotebookLM, dove consegni. Usa parole tue.

6. Come sarai valutato (per sapere cosa conta)

1. **Parte 1 — Git/GitHub (30%)**: risposte corrette a parole tue e screenshot della storia dei commit.
2. **Parte 2 — Gemini con .md (25%)**: esercitazione svolta, screenshot, e hai indicato la versione usata.
3. **Parte 3 — NotebookLM con PDF (25%)**: esercitazione svolta, screenshot, e hai colto la differenza con Gemini.
4. **Domanda finale + parole tue (20%)**: sai spiegare il flusso completo con parole tue (la prova del nove).

> **NOTA**
>
> Conta piu il “saperlo spiegare a parole tue” che il copia-incolla: se sai raccontare cosa hai fatto, hai imparato davvero.

Changelog

1. **v1.0 (29/10/2026)**: prima versione, dalle fonti della lezione.